디지털논리회로설계

VHDL Assignment 3

Latch 및 Flip-Flop 설계

학과	로봇공학과
학번	2023050146
이름	김태수
제출일	2026. 11. 13

1. RS Latch

1.1 동작 원리

NOR 게이트 두 개의 출력을 서로의 입력으로 연결하였다. 식으로 쓰면 Q = NOR(R, Qn), Qn = NOR(S, Q)이다. S=1이면 Q가 1이 되고, R=1이면 Q가 0이 된다. S와 R이 모두 0이면 피드백 때문에 이전 상태가 유지된다. S와 R이 모두 1이면 Q와 Qn이 둘 다 0이 되어 서로 반대 값이라는 조건이 깨지므로 금지 입력으로 둔다. VHDL도 같은 두 개의 NOR 식으로 작성하였다.

표 1. RS Latch 진리표

S	R	Q	Qn	동작
0	0	이전 값	이전 값	Hold
1	0	1	0	Set
0	1	0	1	Reset
1	1	0	0	금지

1.2 VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

-- RS Latch built from two cross-coupled NOR gates
--   S R | Q   Qn
--   0 0 | hold
--   1 0 | 1   0   (Set)
--   0 1 | 0   1   (Reset)
--   1 1 | 0   0   (Forbidden)
entity rs_latch is
    port(
        s  : in  std_logic;
        r  : in  std_logic;
        q  : out std_logic;
        qn : out std_logic
    );
end entity rs_latch;

architecture gate of rs_latch is
    signal q_int  : std_logic;
    signal qn_int : std_logic;
begin
    q_int  <= r nor qn_int;
    qn_int <= s nor q_int;

    q  <= q_int;
    qn <= qn_int;
end architecture gate;
```

1.3 게이트 회로 버전

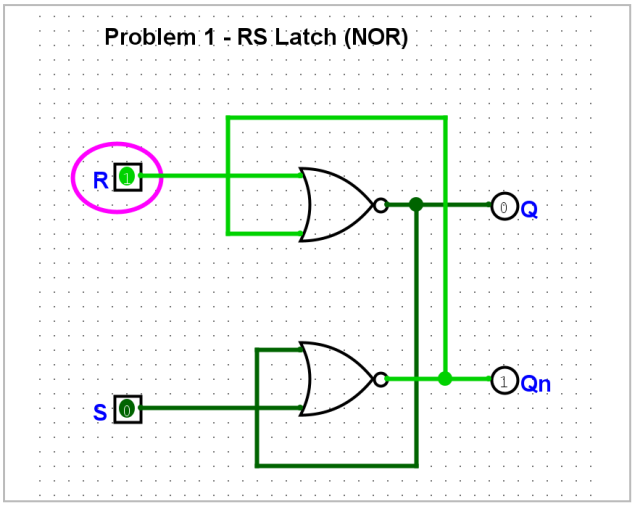


그림 1-1. R=1 → Q=0, Qn=1 (Reset)

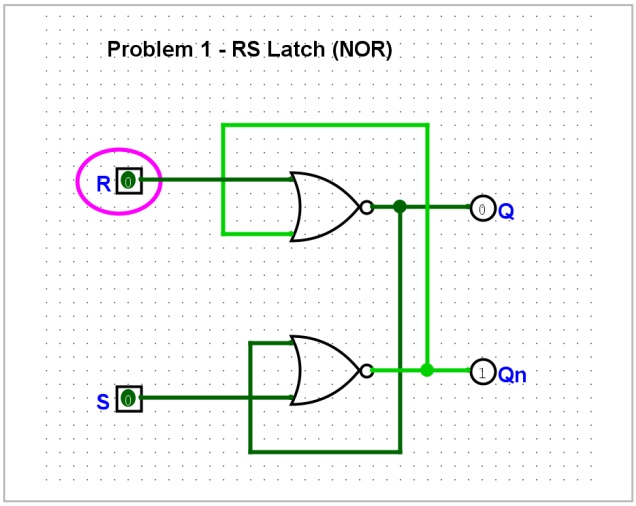


그림 1-2. Reset 후 R=0 → Q=0 유지 (Hold)

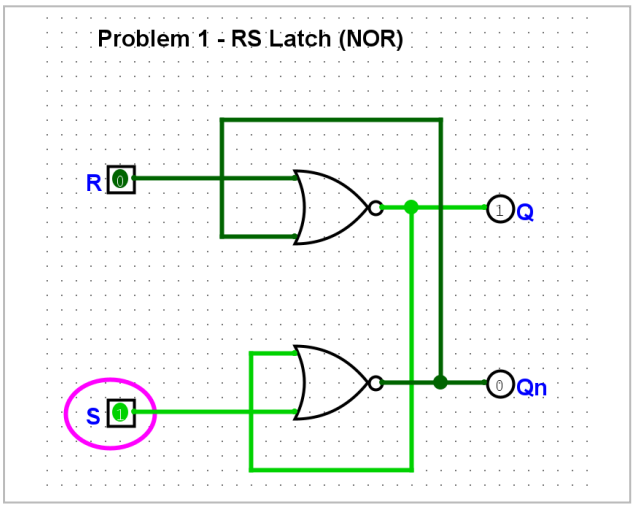


그림 1-3. S=1 → Q=1, Qn=0 (Set)

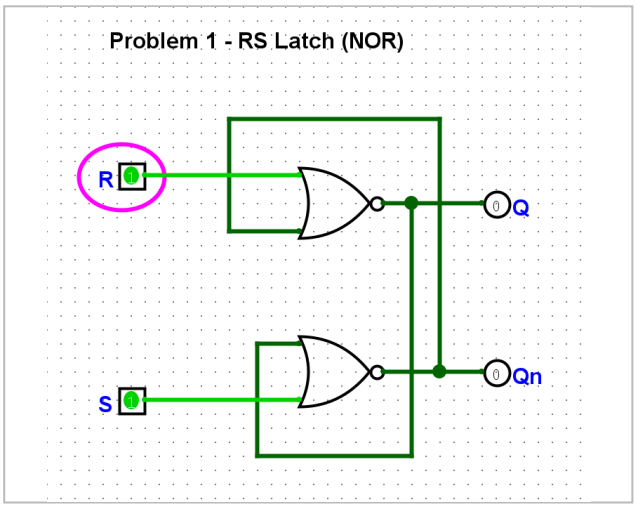


그림 1-4. S=R=1 → Q=Qn=0 (금지 입력)

1.4 VHDL 블랙박스 버전

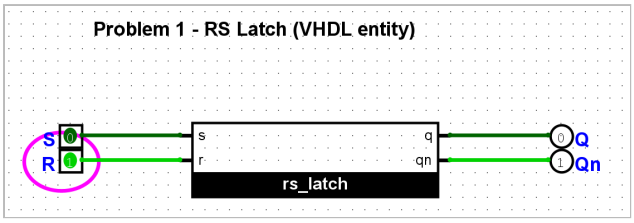


그림 1-5. R=1 → Q=0, Qn=1 (Reset)

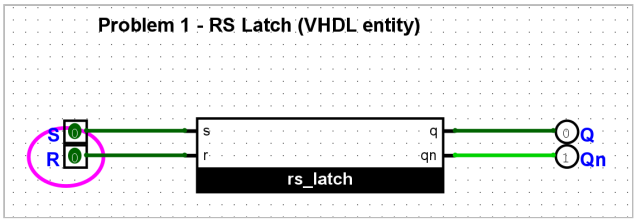


그림 1-6. Reset 후 S=R=0 → Q=0 유지 (Hold)

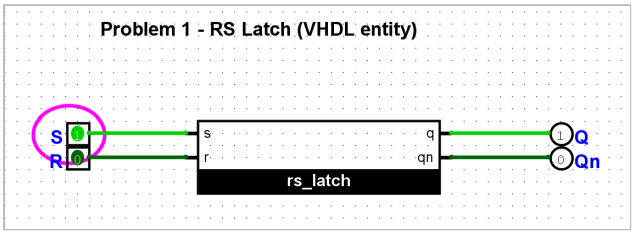


그림 1-7. S=1 → Q=1, Qn=0 (Set)

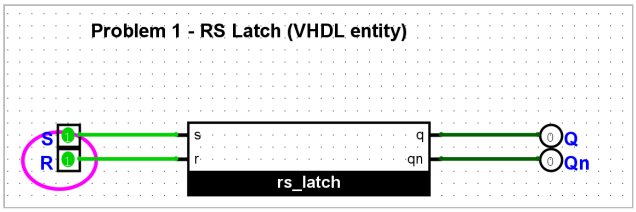


그림 1-8. S=R=1 → Q=Qn=0 (금지 입력)

1.5 비교

네 가지 입력 조건 모두에서 게이트 회로와 VHDL 블랙박스의 Q, Qn이 같았다. 금지 입력에서도 두 버전 모두 Q=Qn=0이 나왔다.

2. D Latch

2.1 동작 원리

NAND 4개로 구성하였다. 앞단의 N1 = NAND(D, en), N2 = NAND(N1, en)이 입력을 만들고, 뒷단의 N3, N4가 NAND형 SR 래치를 이룬다. en=1이면 N1 = D', N2 = D가 되어 Q에 D가 그대로 나타난다. en=0이면 N1과 N2가 모두 1이 되어 래치가 이전 값을 유지한다. 강의자료의 회로는 D를 인버터로 뒤집어 N2에 넣지만, N2의 입력으로 N1의 출력을 쓰면 인버터 없이 같은 동작을 얻을 수 있어 NAND 4개만 사용하였다. VHDL은 en='1'일 때만 q에 d를 넣는 process로 작성하였다.

표 2. D Latch 진리표

en	D	Q	동작
1	0	0	Q가 D를 따라감
1	1	1	Q가 D를 따라감
0	X	이전 값	유지

2.2 VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

-- Gated D Latch
--   en = '1' : Q follows D (transparent)
--   en = '0' : Q holds its previous value
entity d_latch is
    port(
        d  : in  std_logic;
        en : in  std_logic;
        q  : out std_logic;
        qn : out std_logic
    );
end entity d_latch;

architecture bhv of d_latch is
    signal q_int : std_logic := '0';
begin
    process(d, en)
    begin
        if en = '1' then
            q_int <= d;
        end if;
    end process;

    q  <= q_int;
    qn <= not q_int;
end architecture bhv;
```

2.3 게이트 회로 버전

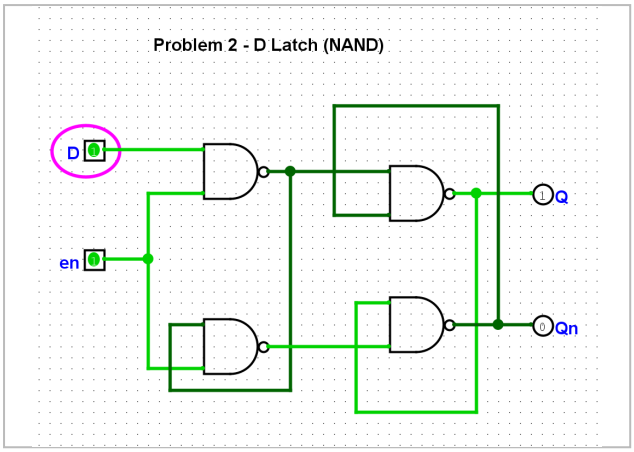


그림 2-1. en=1, D=1 → Q=1

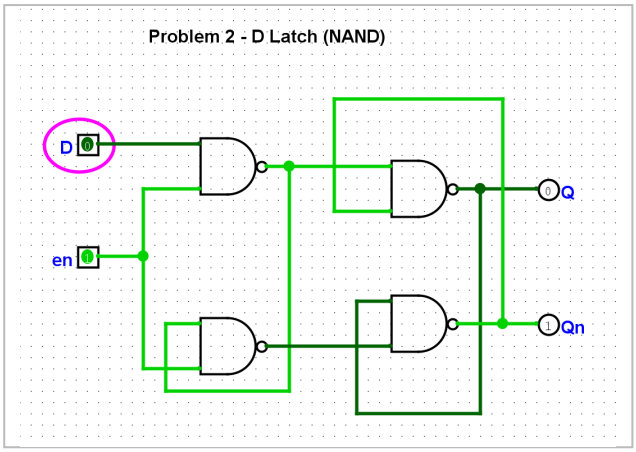


그림 2-2. en=1, D=0 → Q=0

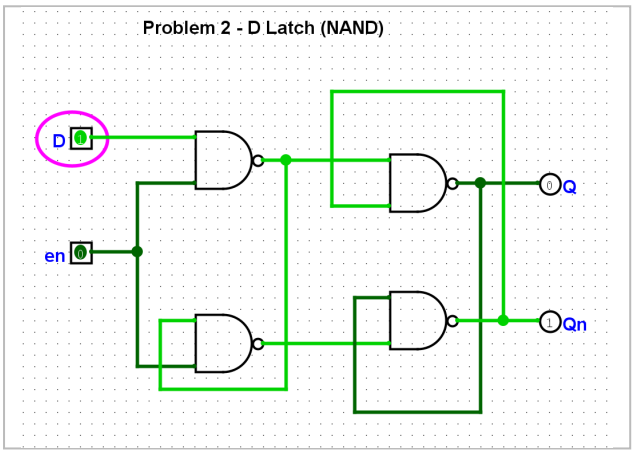


그림 2-3. Q=0 저장 후 en=0, D=1 → Q=0 유지

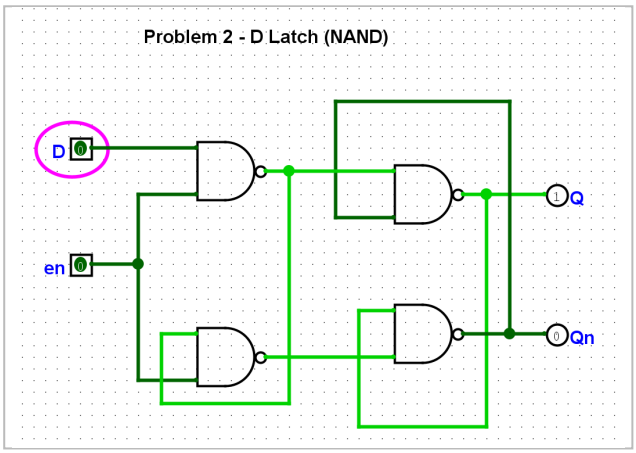


그림 2-4. Q=1 저장 후 en=0, D=0 → Q=1 유지

2.4 VHDL 블랙박스 버전

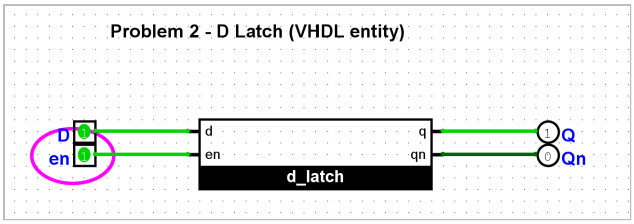


그림 2-5. en=1, D=1 → Q=1

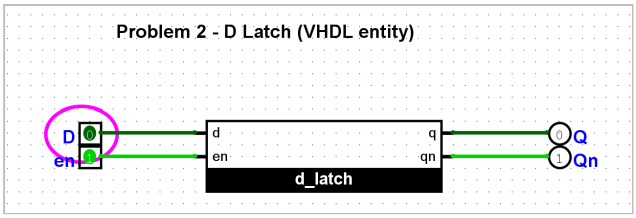


그림 2-6. en=1, D=0 → Q=0

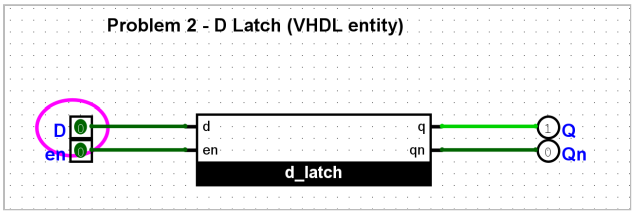


그림 2-7. en=1, D=1로 Q=1 저장 후 en=0, D=0 → Q=1 유지

2.5 비교

en=1일 때 두 버전 모두 Q가 D를 바로 따라갔고, en=0일 때는 D를 바꿔도 Q가 그대로였다.

3. D Flip-Flop

3.1 동작 원리

문제 2의 D Latch 두 개와 인버터 하나로 Master-Slave 구조를 만들었다. Master의 enable에는 clk', Slave의 enable에는 clk를 넣고, Master의 출력을 Slave의 D로 연결하였다. clk=0이면 Master가 D를 받아들이고 Slave는 출력을 유지한다. clk가 0에서 1로 올라가면 Master가 값을 고정하고 Slave가 그 값을 Q로 내보낸다. clk=1인 동안에는 Master가 닫혀 있어 D가 바뀌어도 Q가 변하지 않는다. 따라서 Q는 상승 에지에서만 바뀐다. VHDL은 과제 조건대로 rising_edge(clk)를 사용하였다.

표 3. D Flip-Flop 특성표

clk	D	Q(next)	동작
0→1	0	0	D 저장
0→1	1	1	D 저장
그 외	X	Q	유지

3.2 VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

-- Positive-edge-triggered D Flip-Flop
-- Q is updated only on the rising edge of clk.
entity d_ff is
    port(
        d   : in  std_logic;
        clk : in  std_logic;
        q   : out std_logic;
        qn  : out std_logic
    );
end entity d_ff;

architecture bhv of d_ff is
    signal q_int : std_logic := '0';
begin
    process(clk)
    begin
        if rising_edge(clk) then
            q_int <= d;
        end if;
    end process;

    q  <= q_int;
    qn <= not q_int;
end architecture bhv;
```

3.3 게이트 회로 버전

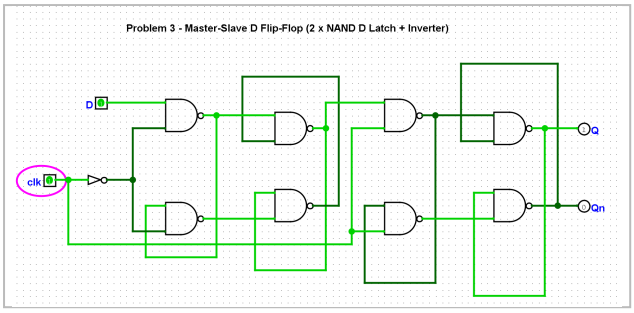


그림 3-1. D=1에서 clk 0→1 → Q=1

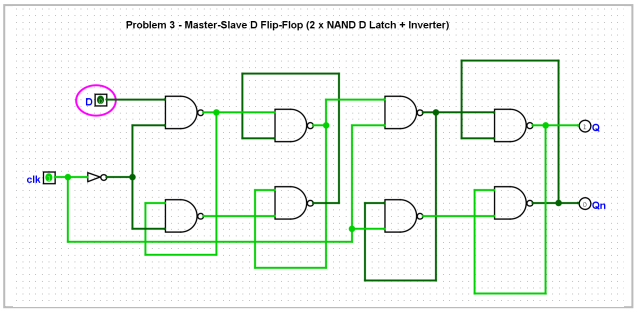


그림 3-2. clk=1인 상태에서 D=0 → Q=1 유지

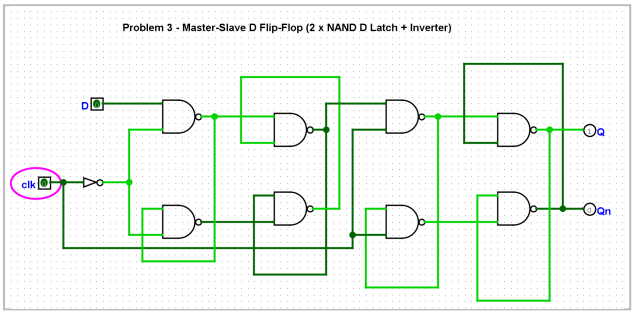


그림 3-3. clk=0 (Master만 D=0을 받음) → Q=1 유지

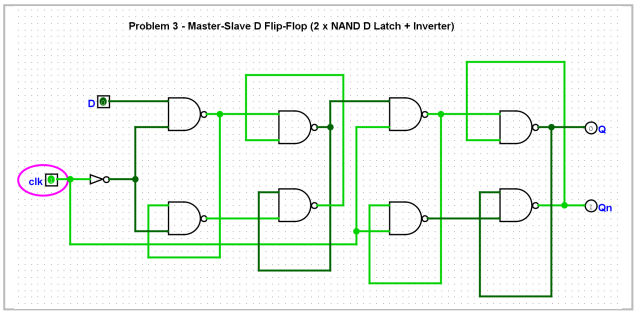


그림 3-4. clk 0→1 → Q=0

3.4 VHDL 블랙박스 버전

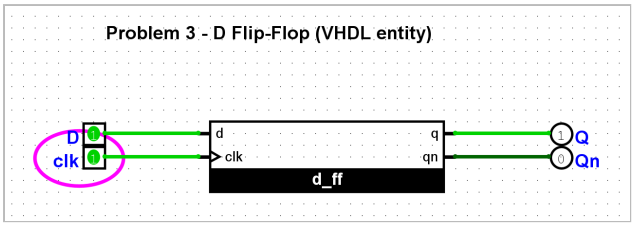


그림 3-5. D=1에서 clk 0→1 → Q=1

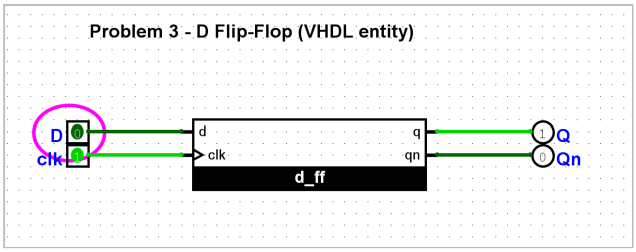


그림 3-6. 상승 에지 후 clk=1인 상태에서 D=0 → Q=1 유지

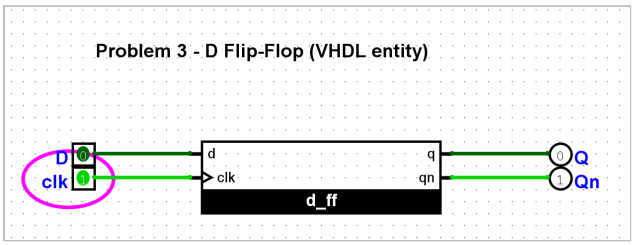


그림 3-7. clk 1→0→1 → Q=0

3.5 비교

두 버전 모두 clk가 올라가는 순간에만 Q가 바뀌었고, clk가 1로 유지되는 동안 D를 바꾸어도 Q는 변하지 않았다.

4. JK Flip-Flop

4.1 동작 원리

JK Flip-Flop의 다음 상태는 Q(next) = J·Qn + K'·Q이다. 이 식을 NAND로 만들어(a = NAND(J, Qn), b = NAND(K', Q), D = NAND(a, b)) 상승 에지 D Flip-Flop의 D 입력에 연결하였다. 출력 Q와 Qn이 다시 입력 쪽 NAND로 돌아가는 피드백 구조이다. D Flip-Flop 부분은 NAND 6개(G1~G6)로 된 에지 트리거 구조이다.

표 4. JK Flip-Flop 특성표 (clr=0)

J	K	Q(next)	동작
0	0	Q	Hold
0	1	0	Reset
1	0	1	Set
1	1	Q'	Toggle

4.2 VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

-- Positive-edge-triggered JK Flip-Flop (with asynchronous clear for initialization)
--   clr = '1' : Q = '0' immediately
--   J K | Q(next) on rising edge of clk
--   0 0 | Q      (Hold)
--   0 1 | 0      (Reset)
--   1 0 | 1      (Set)
--   1 1 | not Q  (Toggle)
entity jk_ff is
    port(
        j   : in  std_logic;
        k   : in  std_logic;
        clk : in  std_logic;
        clr : in  std_logic;
        q   : out std_logic;
        qn  : out std_logic
    );
end entity jk_ff;

architecture bhv of jk_ff is
    signal q_int : std_logic := '0';
begin
    process(clk, clr)
        variable jk : std_logic_vector(1 downto 0);
    begin
        if clr = '1' then
            q_int <= '0';
        elsif rising_edge(clk) then
            jk := j & k;
            case jk is
                when "00"   => q_int <= q_int;      -- Hold
                when "01"   => q_int <= '0';        -- Reset
                when "10"   => q_int <= '1';        -- Set
                when "11"   => q_int <= not q_int;  -- Toggle
                when others => q_int <= q_int;
            end case;
        end if;
    end process;

    q  <= q_int;
    qn <= not q_int;
end architecture bhv;
```

4.3 게이트 회로 버전

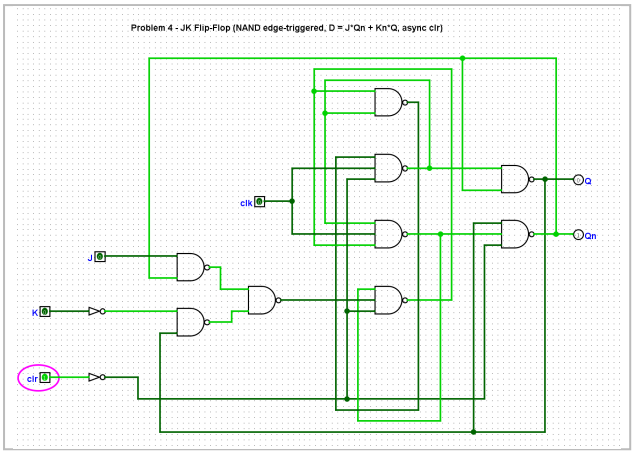


그림 4-1. clr=1 → Q=0 (초기화)

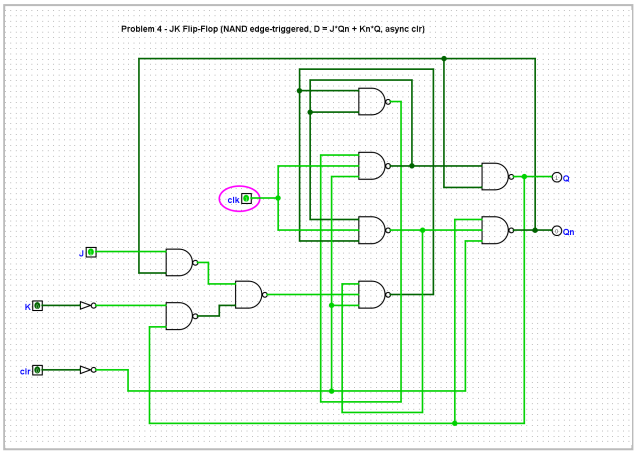


그림 4-2. J=1, K=0, clk↑ → Q=1 (Set)

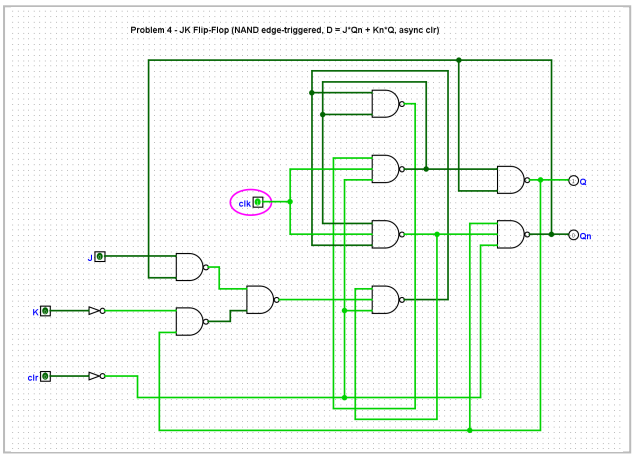


그림 4-3. J=0, K=0, clk↑ → Q=1 유지 (Hold)

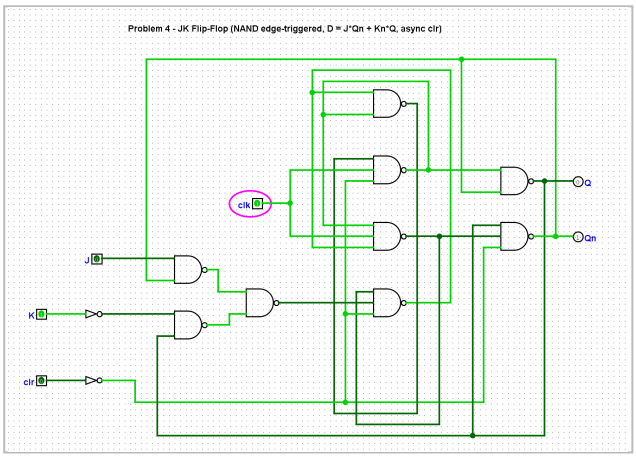


그림 4-4. J=0, K=1, clk↑ → Q=0 (Reset)

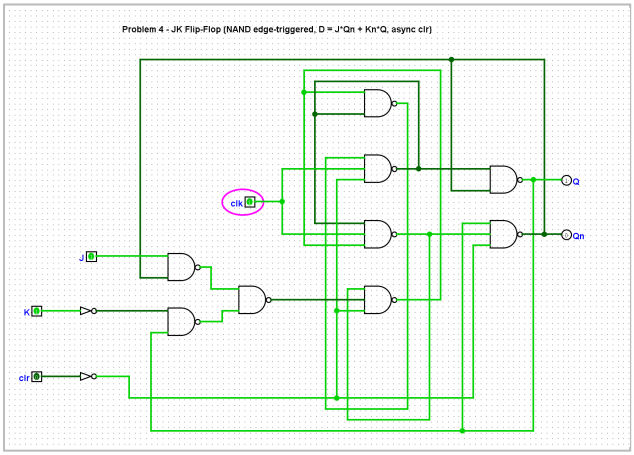


그림 4-5. J=K=1, clk↑ → Q=1 (Toggle)

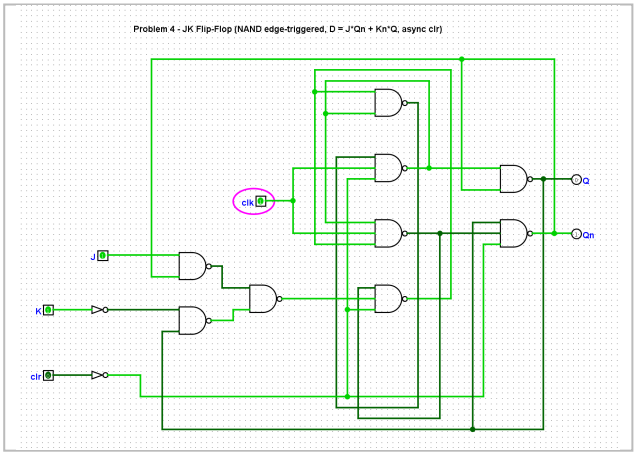


그림 4-6. J=K=1, clk↑ → Q=0 (Toggle)

4.4 VHDL 블랙박스 버전

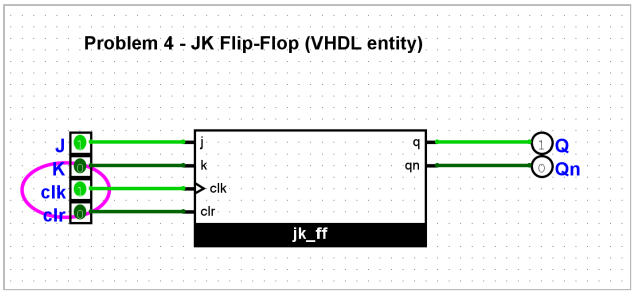


그림 4-7. J=1, K=0, clk↑ → Q=1 (Set)

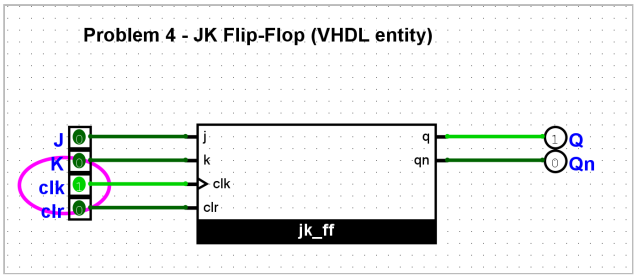


그림 4-8. Set 후 J=0, K=0, clk↑ → Q=1 유지 (Hold)

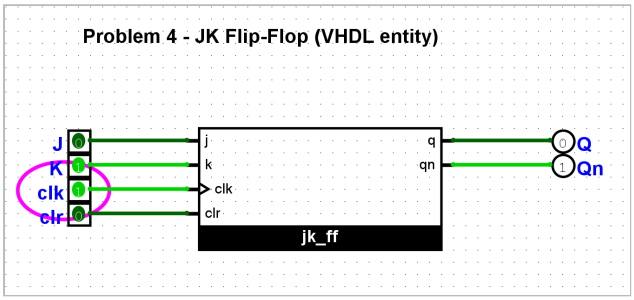


그림 4-9. J=0, K=1, clk↑ → Q=0 (Reset)

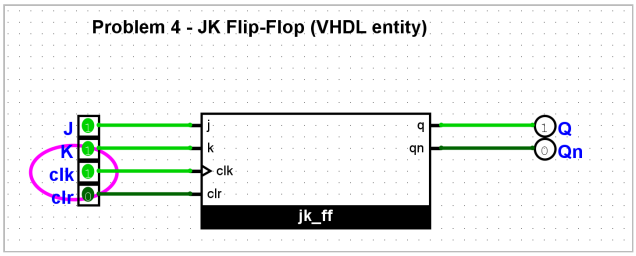


그림 4-10. Reset 후 J=K=1, clk↑ → Q=1 (Toggle)

4.5 비교

Hold, Set, Reset, Toggle 네 동작 모두 두 버전의 결과가 같았다. 게이트 회로에서는 J=K=1에서 clk를 올릴 때마다 Q가 1, 0으로 번갈아 바뀌는 것을 확인하였다.

4.6 설계 시 고려한 점

Logisim에서 피드백 회로는 처음 열었을 때 Q, Qn 값이 정해지지 않아 스스로 빠져나오지 못한다. 그래서 강의자료의 JK 코드처럼 비동기 clr 입력을 추가하였다. rn = NOT(clr)을 G2, G4, G6에 넣어 clr=1이면 Q=0이 된다. VHDL에도 같은 clr을 넣었다. 강의자료의 Master-Slave JK는 clk=0인 동안 J, K의 짧은 변화를 잡아 버리는 현상이 있어 rising_edge로 작성한 VHDL과 결과가 달라질 수 있으므로, 에지 트리거 구조를 사용하였다.

5. D Flip-Flop with Asynchronous Set/Clear

5.1 동작 원리

NAND 6개로 된 에지 트리거 D Flip-Flop(74LS74와 같은 구조)에 비동기 입력을 추가하였다. rn = NOT(clr)을 G2, G4, G6에, sn = NAND(set, rn)을 G1, G5에 연결하였다. clr=1이면 rn=0이 되어 클럭과 관계없이 Q=0이 된다. 이때 sn = NAND(set, 0) = 1이 되므로 set은 무시되고, 이렇게 해서 Clear가 Set보다 우선한다. clr=0, set=1이면 sn=0이 되어 Q=1이 된다. 둘 다 0이면 일반 D Flip-Flop으로 동작한다. VHDL에서는 process 안의 if 순서를 clr, set, rising_edge 순으로 두어 같은 우선순위를 만들었다.

표 5. 비동기 Set/Clear D Flip-Flop 동작표

clr	set	clk	Q	동작
1	X	X	0	Clear (최우선)
0	1	X	1	Set
0	0	0→1	D	일반 D FF
0	0	그 외	Q	유지

5.2 VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

-- Positive-edge-triggered D Flip-Flop with asynchronous Set and Clear
--   clr = '1'           : Q = '0' immediately (highest priority)
--   set = '1', clr = '0': Q = '1' immediately
--   set = clr = '0'     : normal D Flip-Flop (Q <= D on rising edge of clk)
entity d_ff_sc is
    port(
        d   : in  std_logic;
        clk : in  std_logic;
        set : in  std_logic;
        clr : in  std_logic;
        q   : out std_logic;
        qn  : out std_logic
    );
end entity d_ff_sc;

architecture bhv of d_ff_sc is
    signal q_int : std_logic := '0';
begin
    process(clk, set, clr)
    begin
        if clr = '1' then
            q_int <= '0';
        elsif set = '1' then
            q_int <= '1';
        elsif rising_edge(clk) then
            q_int <= d;
        end if;
    end process;

    q  <= q_int;
    qn <= not q_int;
end architecture bhv;
```

5.3 게이트 회로 버전

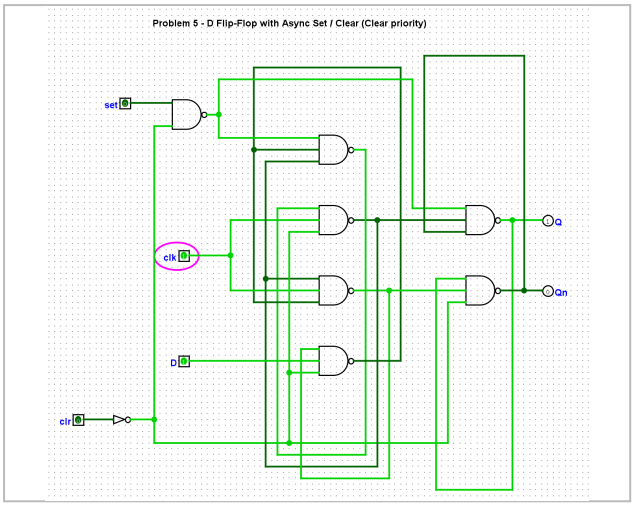


그림 5-1. D=1, clk↑ → Q=1

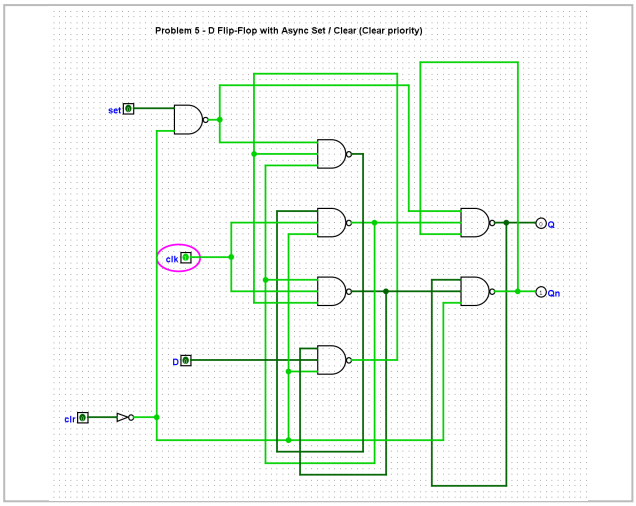


그림 5-2. D=0, clk↑ → Q=0

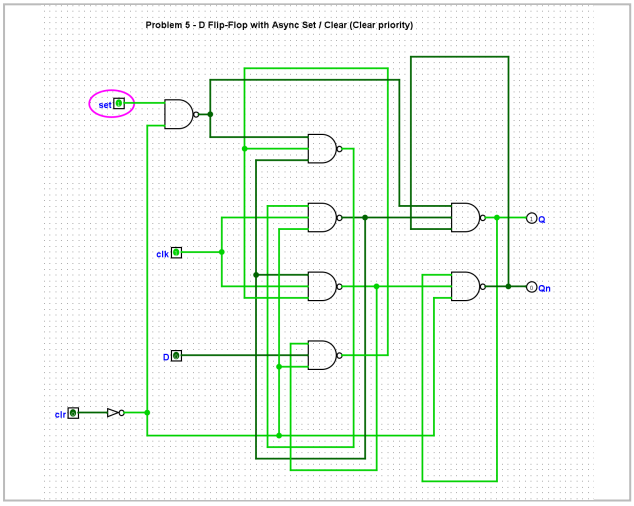


그림 5-3. set=1 → Q=1 (클럭과 무관)

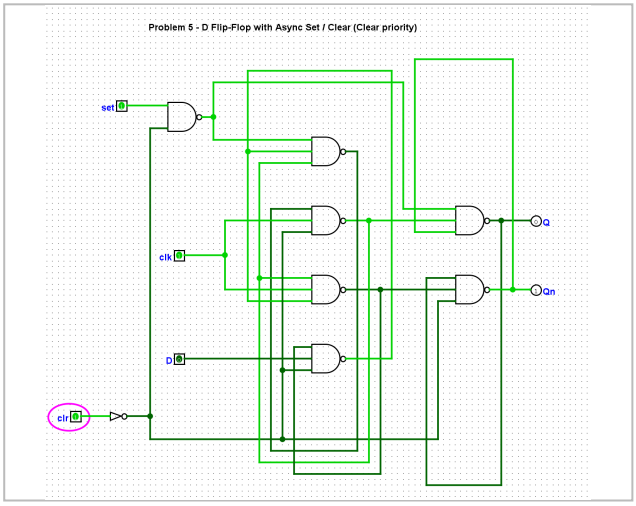


그림 5-4. set=1, clr=1 → Q=0 (Clear 우선)

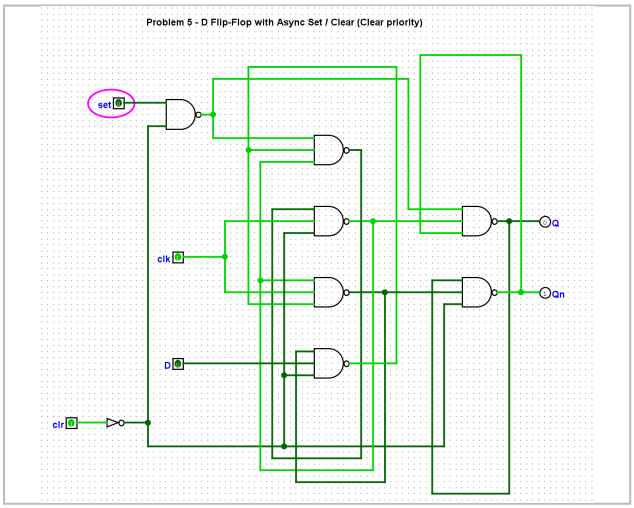


그림 5-5. clr=1 → Q=0

5.4 VHDL 블랙박스 버전

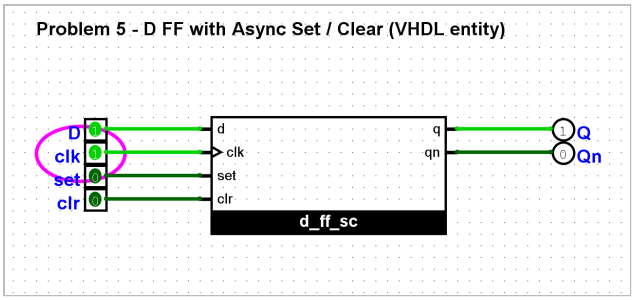


그림 5-6. D=1, clk↑ → Q=1

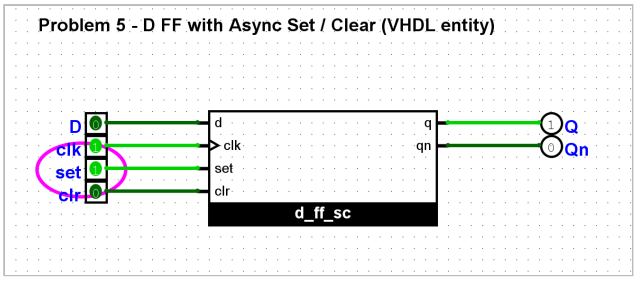


그림 5-7. D=0이지만 set=1 → Q=1

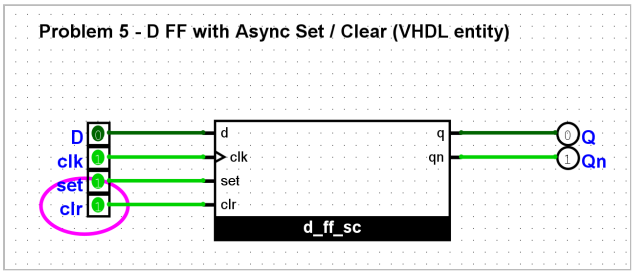


그림 5-8. set=1, clr=1 → Q=0 (Clear 우선)

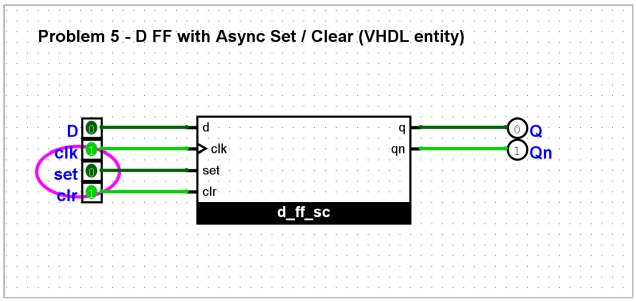


그림 5-9. clr=1 → Q=0

5.5 비교

일반 동작, 비동기 Set, 비동기 Clear, Set과 Clear 동시 입력의 네 경우 모두 두 버전의 결과가 같았다. 동시 입력에서는 두 버전 모두 Q=0으로 Clear가 우선하였다.

사용 도구 및 제출 파일

Logisim-evolution 5.0.0으로 회로를 구성하였고, VHDL Entity 시뮬레이션은 Logisim과 연동한 Questa Altera Starter FPGA Edition으로 수행하였다. 캡처에서 밝은 초록색 선은 1, 어두운 초록색 선은 0이다.

폴더	내용
problem1 ~ 5	*.vhd (VHDL 코드), *.circ (게이트 회로), *_vhdl.circ (VHDL 블랙박스)
problemN/screenshots	게이트 회로 캡처 (P*_logisim_*), 블랙박스 캡처 (P*_vhdl_*)